encamina
PIENSA EN COLORES

# Guía paso a paso: constituciones Spec Kit

## Anexo A del taller greenfield | Open Finance

| | |
|---|---|
| **Documento** | Anexo A del taller greenfield |
| **Contenido** | Constituciones Spec Kit: corporativa, de dominio y de API |
| **Contexto** | Open Finance / GitLab / Spec Kit / Jenkins |
| **Formato** | Guía paso a paso |




**ENCAMINA**

Octubre 2026

**Valencia (sede central)**
Av. Jerónimo Roure, 49
46520, Sagunto (Valencia)
Telf. +34 962 698 064

**Madrid**
C/ O'Donnell, 34
28009, Madrid
Telf. +34 917 893 823

**Murcia**
C/ del Ángel, 32
30202 Cartagena (Murcia)

**Tenerife**
Av. Trinidad, 61. Campus Central ULL
38200, S. Cristóbal de La Laguna (Tenerife)

info@encamina.com
www.encamina.com

# Guía paso a paso: constituciones Spec Kit

## Índice

1 Anexo A. Construcción y especialización de constituciones ..... 3
1.1 Regla de dependencia ..... 3
1.2 Un proyecto Spec Kit por repositorio ..... 3
1.3 Ejemplo 1. Constitución corporativa ..... 4
1.4 Ejemplo 2. Especialización de la constitución del dominio ..... 4
1.5 Ejemplo 3. Especialización del perfil de una API ..... 5
1.6 Composición en cada proyecto consumidor ..... 6
1.7 Guía para implementar la pipeline de Jenkins ..... 7
1.8 Comprobación rápida de dependencia ..... 10

# 1 Anexo A. Construcción y especialización de constituciones

Este anexo muestra cómo redactar las fuentes de gobierno y obtener la constitución efectiva que utilizará Spec Kit en cada repositorio. Los identificadores y textos son ejemplos para el taller; deberán aprobarse y adaptarse antes de convertirse en reglas corporativas o de dominio.

## 1.1 Regla de dependencia

La cadena de gobierno tiene tres niveles y una sola dirección:

```
Constitución corporativa
          ↓ completa y obligatoria
Constitución del dominio
          ↓ completa y obligatoria
Perfil de especialización de la API
          ↓ se compone, no se hereda automáticamente
Constitución efectiva del repositorio de la API
```

La constitución corporativa define el suelo común. La constitución del dominio incorpora ese suelo y concreta las reglas que corresponden a su negocio. El perfil de la API incorpora ambos niveles y declara solo las decisiones propias de esa API. Un nivel inferior puede añadir restricciones o concretar una opción que el superior haya dejado abierta, pero no puede eliminar, debilitar ni reemplazar una obligación superior. Si dos reglas son incompatibles, la composición se detiene hasta que el responsable de la regla superior resuelva el conflicto.

## 1.2 Un proyecto Spec Kit por repositorio

Cada repositorio es un proyecto de trabajo independiente de Spec Kit. Tener los repositorios relacionados en el mismo grupo de GitLab, clonar sus fuentes o mencionar sus rutas no hace que Spec Kit cargue sus constituciones. El contexto requerido debe estar versionado y disponible en el repositorio en el que se ejecuta Spec Kit.

| Repositorio | Fuente de gobierno que mantiene | Constitución disponible para su propio Spec Kit |
|---|---|---|
| `api-standards` | Constitución corporativa aprobada. | La constitución corporativa, que es la fuente raíz y no depende de una constitución superior de Open Finance. |
| `domain-governance` | Perfil o capítulos de especialización del dominio y referencia fijada a la constitución corporativa. | Constitución efectiva del dominio, compuesta por la corporativa y la especialización del dominio. |
| Proyecto de una API | `governance/api-profile.md` y referencias fijadas a las fuentes corporativa y de dominio. | Constitución efectiva de esa API, compuesta por las reglas corporativas, las del dominio y el perfil local. |

Cada proyecto tendrá su configuración local `.specify/`, su contexto y sus artefactos `specs/`. En cada uno, `.specify/memory/constitution.md` será el documento completo que utilizará Spec Kit. En el repositorio corporativo puede ser directamente la fuente corporativa; en los repositorios de dominio y API será un resultado generado y revisable. La composición deberá ejecutarse localmente antes de usar Spec Kit y reproducirse en Jenkins. No se dará por supuesto que Spec Kit resuelve referencias remotas, combina constituciones o aplica herencia por sí solo.

## 1.3 Ejemplo 1. Constitución corporativa

La constitución corporativa debe contener reglas estables que apliquen a todas las APIs de Open Finance, con identificadores que permitan citarlas sin copiar párrafos en cada nivel. Cada capítulo deberá indicar su alcance, su obligación, las decisiones que deja abiertas y quién puede aprobar una excepción.

Por ejemplo, una primera versión podría definir estos capítulos:

| Identificador de ejemplo | Regla corporativa | Qué puede concretar un nivel inferior |
|---|---|---|
| `OF-SCOPE-01` | El trabajo gobernado por esta constitución produce especificaciones y contratos OpenAPI; no genera la implementación del backend. | La API identifica su capacidad, su capa API-Led y los artefactos contractuales que le corresponden. |
| `OF-CLARITY-01` | Antes de generar artefactos se resuelven las ambigüedades relevantes conforme a la regla de confianza mínima del 95 %; no se inventan requisitos funcionales. | El dominio y la API indican qué responsables deben aprobar sus decisiones de negocio. |
| `OF-MODEL-01` | Se reutilizan modelos canónicos aprobados cuando cubren la necesidad; no se copian ni se alteran silenciosamente. | El dominio identifica sus modelos aprobados y la API selecciona versiones y usos concretos. |
| `OF-SECURITY-01` | La seguridad y los permisos aplicables deben estar definidos antes de aprobar el contrato. | El dominio y la API concretan los mecanismos y permisos aprobados para sus consumidores y operaciones. |
| `OF-QUALITY-01` | El contrato debe superar las validaciones corporativas obligatorias y conservar evidencias. | Los niveles inferiores pueden añadir comprobaciones, pero no desactivar las obligatorias. |

Ejemplo de redacción de un capítulo corporativo:

> **OF-MODEL-01 — Reutilización de modelos canónicos.** Las APIs DEBEN utilizar un modelo canónico aprobado cuando represente la información requerida. No DEBEN duplicar ni modificar localmente una definición compartida para evitar el proceso de gobierno. Si no existe un modelo adecuado, el equipo DEBE solicitar su definición o evolución al responsable del dominio antes de aprobar el contrato. La constitución de dominio puede identificar modelos y restricciones del dominio; no puede autorizar una copia local que contradiga esta regla.

Al aprobar la constitución corporativa, el equipo publicará una revisión identificable en `api-standards`. Sus capítulos tendrán responsables y un proceso de excepción explícito. La revisión usada por un consumidor se fijará mediante el commit aprobado, no mediante una referencia móvil.

## 1.4 Ejemplo 2. Especialización de la constitución del dominio

La constitución de dominio no vuelve a redactar la política corporativa ni constituye una alternativa a ella. Como vive en otro repositorio, no depende de una ruta relativa que salga del repositorio ni de que Spec Kit encuentre el proyecto corporativo. El repositorio de dominio registra la fuente corporativa por su ruta de proyecto GitLab, la revisión inmutable aprobada y el identificador estable del capítulo que concreta. El proceso de composición obtiene esa revisión y hace disponible su contenido localmente; el ID sirve para mantener la trazabilidad, no es una referencia que Spec Kit resuelva por sí solo.

En el repositorio `domain-governance`, el registro de fuentes de gobierno debe dejar constancia de:

| Dato | Registro para este ejemplo |
|---|---|
| Fuente corporativa | Proyecto GitLab `openfinance/platform/api-standards`. |
| Revisión adoptada | SHA del commit aprobado que el responsable del dominio decidió consumir; se registra el valor real, no una rama como `main`. |
| Capítulo corporativo | `OF-MODEL-01`, cuyo texto se obtiene de esa revisión. |
| Capítulo de dominio | `PORT-MODEL-01`, que declara explícitamente que especializa `OF-MODEL-01`. |
| Resultado para Spec Kit | Constitución efectiva del dominio con el texto corporativo completo y la regla de dominio, junto con la procedencia de ambos. |

Así, si `api-standards` y `domain-governance` están en repositorios separados, el vínculo se conserva en los archivos versionados del dominio como una referencia a proyecto + commit + ID de capítulo. La revisión de origen no se adivina ni se actualiza automáticamente. El proceso local debe poder recuperar ese commit —por una clonación de la fuente o el mecanismo versionado aprobado— antes de ejecutar Spec Kit; Jenkins debe repetir la misma composición.

Para `portfolio-management`, el registro anterior relaciona el capítulo corporativo con su especialización local. El texto de `PORT-MODEL-01` se mantiene en el repositorio de dominio y podría establecer:

> **PORT-MODEL-01 — Modelos de cartera.** En el dominio `portfolio-management`, los conceptos de cartera y posición se representan mediante los modelos canónicos aprobados para ese dominio. La revisión semántica y la ruta del modelo se seleccionan explícitamente por contrato. Una API que necesite atributos que esos modelos no contienen debe justificar una representación específica o solicitar una evolución del modelo; no debe modificar el golden record compartido. Este capítulo concreta `OF-MODEL-01` y no crea una excepción a él.

La constitución del dominio también puede añadir una regla nueva, siempre que respete las obligaciones corporativas. Por ejemplo, puede definir quién aprueba la terminología de cartera, cómo se determina la propiedad de un modelo o qué revisión de un modelo está aprobada para nuevos contratos. No debe declarar por sí sola una política de agregación de inversiones si esa decisión funcional todavía no está aprobada.

El proyecto `domain-governance` versionará la constitución del dominio, el registro de su fuente exacta en `api-standards` y la evidencia de revisión. Al componer `.specify/memory/constitution.md` dentro del repositorio de dominio, se incorporará el texto íntegro de `OF-MODEL-01` desde el commit fijado, seguido de `PORT-MODEL-01` y su relación explícita. De este modo, Spec Kit trabaja con un documento completo que está en su propio repositorio, mientras que el registro permite rastrear qué capítulo y revisión corporativos lo originaron. Si no se puede recuperar esa fuente o el capítulo no existe en la revisión fijada, la composición falla y no se genera la constitución efectiva.

## 1.5 Ejemplo 3. Especialización del perfil de una API

El perfil de API no es una tercera constitución independiente. Es un documento breve que identifica las particularidades del proyecto y referencia las reglas compartidas aplicables. Para `of-prc-port-investment-summary-v1`, podría registrar:

| Dato del perfil | Valor del caso del taller |
|---|---|
| API y capa | `of-prc-port-investment-summary-v1`, Process API. |
| Dominio propietario | `portfolio-management`. |
| Propósito | Coordinar la exposición contractual del resumen de inversiones, según los requisitos aprobados. |
| Capítulos corporativos aplicables | `OF-SCOPE-01`, `OF-CLARITY-01`, `OF-MODEL-01`, `OF-SECURITY-01` y `OF-QUALITY-01`. |
| Capítulos del dominio aplicables | `PORT-MODEL-01` y los capítulos de dominio aprobados que correspondan a la capacidad. |
| Decisiones locales | Seleccionar los modelos y sus versiones aprobadas que requiera el contrato; indicar sus operaciones y consumidores solo cuando estén definidos por la especificación aprobada. |

El perfil no puede decidir por sí solo rutas, permisos, campos, reglas de agregación, tratamiento de datos parciales o respuestas de error que no estén aprobados. Esas decisiones pertenecen a la especificación funcional de la API. Si el perfil necesita una regla que contradice una fuente compartida, se detiene la generación y se solicita la resolución al responsable correspondiente.

Cada proyecto de Experience, Process y System tendrá su propio perfil y su propia constitución efectiva. El perfil de una API no se reutilizará como si gobernara automáticamente las APIs a las que consume o que la consumen.

## 1.6 Composición en cada proyecto consumidor

El proceso de composición deberá ser reproducible y producir el mismo documento en el entorno local y Jenkins:

1. Identificar el commit aprobado de `api-standards` y, para una API, el commit aprobado del proyecto `domain-governance` de su dominio propietario. Registrar también la revisión del perfil local.
2. Leer las fuentes fijadas y comprobar que todos los identificadores referenciados existen y están vigentes. No sustituir una fuente inaccesible por una copia no aprobada ni por una rama móvil.
3. Incorporar el texto completo de los capítulos corporativos y de dominio aplicables, y después las decisiones locales compatibles. Conservar los identificadores y la procedencia de cada regla.
4. Comprobar que ninguna regla inferior elimina o debilita una obligación superior. Ante una contradicción, no generar ni actualizar la constitución efectiva hasta resolverla mediante el proceso de gobierno.
5. Escribir y revisar `.specify/memory/constitution.md` en el repositorio consumidor. El documento debe incluir las reglas completas, las revisiones de origen y el perfil utilizado; no debe depender de enlaces remotos para que Spec Kit conozca sus obligaciones.
6. Ejecutar en ese mismo repositorio las actividades de Spec Kit solo después de comprobar que la constitución efectiva está disponible y coincide con las fuentes fijadas. Jenkins reproducirá la composición y verificará que el archivo versionado coincide con el resultado.

El resultado es una constitución efectiva por repositorio de dominio y por proyecto de API, no una copia que se actualiza automáticamente cuando cambia una fuente. Cuando se aprueba una nueva revisión corporativa o de dominio, cada consumidor la adopta mediante su propio merge request, regenera su constitución, revisa el impacto y valida sus artefactos antes de continuar.

## 1.7 Guía para implementar la pipeline de Jenkins

Jenkins automatizará la comprobación reproducible del gobierno y de los artefactos de cada proyecto Spec Kit. No reemplazará el trabajo colaborativo de analistas y arquitectos: las decisiones funcionales y las fases que requieran interacción o generación asistida se realizarán en el proyecto correspondiente, se revisarán y se versionarán antes de que la pipeline las valide. Jenkins no completará requisitos pendientes ni aprobará excepciones.

La definición de la pipeline se mantendrá en un proyecto GitLab dedicado a automatización Jenkins, propiedad del equipo de plataforma/DevOps y con una ruta que la organización deberá acordar. No se ubicará en cada proyecto de API ni en `domain-governance`: esos repositorios contienen el contexto propio de Spec Kit y los artefactos que se validan. El repositorio central conservará el `Jenkinsfile` reutilizable y, si se adopta, la librería compartida. Los jobs, credenciales y webhooks se configurarán en Jenkins conforme a la administración de la plataforma; no se generará configuración de GitLab CI/CD.

| Elemento | Ubicación y responsabilidad |
|---|---|
| `Jenkinsfile` y lógica reutilizable | Proyecto central de automatización Jenkins, con revisiones aprobadas y versionadas. No se fijará a una rama móvil para ejecuciones reproducibles. |
| Especificación, constitución efectiva y contrato | En el repositorio de cada proyecto Spec Kit: `api-standards`, `domain-governance` o el proyecto individual de API. |
| Job o ejecución de Jenkins | Un job por repositorio consumidor, o una configuración multibranch equivalente, que obtiene la definición aprobada del proyecto central y valida el commit del repositorio consumidor. La instalación debe configurar esta asociación explícitamente. |
| Credenciales y notificaciones | En Jenkins y en la integración SCM aprobada, con acceso mínimo de solo lectura para recuperar fuentes y reportar el resultado. No se guardan secretos en los repositorios. |

Por tanto, cada API sigue teniendo su propio proyecto Spec Kit, pero no necesita mantener una copia independiente del `Jenkinsfile`. Jenkins ejecuta el flujo central contra el commit de esa API y sus revisiones fijadas de gobierno. El proyecto central y su versión de pipeline también se registrarán en las evidencias de la ejecución.

### 1.7.1 Cuándo se ejecuta

La política concreta de ramas y webhooks debe acordarse con la administración de GitLab y Jenkins. Como mínimo, se recomienda este ciclo:

| Evento | Momento y alcance |
|---|---|
| Merge request abierto, actualizado o reabierto en un repositorio de gobierno o API | Ejecutar las comprobaciones del tipo de repositorio sobre el commit propuesto. El resultado bloqueante debe estar disponible antes de aprobar e integrar el cambio. En una API incluye composición de gobierno y validación del contrato. |
| Cambio integrado en la rama protegida | Volver a validar el commit integrado y conservar sus evidencias. Así, el resultado corresponde exactamente al contenido que quedó en la rama. |
| Nueva API o repositorio de dominio | Ejecutar la validación inicial en su primer merge request, después de registrar sus fuentes, submódulos y configuración local de Spec Kit. No empezar la generación contractual hasta que la composición de gobierno pase. |

| Evento | Momento y alcance |
|---|---|
| Nueva revisión corporativa o de dominio | Ejecutar la pipeline del repositorio que publicó la revisión. Cada consumidor ejecutará la suya cuando proponga adoptar esa revisión en su propio merge request; no se propagan cambios automáticamente. |
| Etiqueta o liberación de una versión de contrato aprobada | Ejecutar la validación completa, generar y volver a validar el consolidado, y archivar el artefacto con su versión, commits y huella. La publicación en el gestor de APIs sigue siendo otra pipeline y requiere sus propias aprobaciones. |

La pipeline no se ejecuta por cada interacción de Spec Kit en el entorno local ni debe iniciar automáticamente una sesión de generación asistida. Se ejecuta cuando hay un cambio versionado que revisar y, para una liberación, cuando se solicita producir el artefacto aprobado. Una comprobación periódica para informar de consumidores que siguen referencias antiguas puede añadirse si se acuerda, pero no modificará repositorios ni sustituirá sus merge requests.

Cada ejecución recibirá el commit del proyecto consumidor que se revisa. La misma definición central podrá distinguir el tipo de repositorio —`api-standards`, `domain-governance` o proyecto de API— y aplicar únicamente las comprobaciones pertinentes.

| Etapa | Trabajo de Jenkins | Condición de bloqueo |
|---|---|---|
| Preparar ejecución | Recuperar el commit exacto del repositorio revisado y verificar el tipo de proyecto y sus archivos requeridos (`.specify/`, perfil y fuentes de gobierno aplicables). | No se puede identificar el proyecto, su commit o sus requisitos locales. |
| Resolver fuentes | Recuperar las revisiones fijadas de la constitución corporativa y, para dominio y API, las fuentes adicionales declaradas. Registrar proyecto y commit de cada fuente; no resolver referencias desde ramas móviles. | Una fuente no se puede recuperar, el commit no existe o un capítulo citado no está presente. |
| Componer gobierno | En los proyectos de dominio y API, ejecutar el mecanismo de composición aprobado para producir la constitución efectiva local. En `api-standards`, validar la constitución corporativa fuente. | Hay referencias rotas, reglas duplicadas incompatibles o una composición no reproducible. |
| Verificar constitución | Comparar el resultado con `.specify/memory/constitution.md` versionado y comprobar su procedencia. Revisar que los capítulos aplicables estén completos y que no se hayan debilitado reglas superiores. | El resultado no coincide, faltan fuentes o se detecta un conflicto sin resolver. |
| Verificar artefactos Spec Kit | Comprobar la configuración y los artefactos esperados del repositorio conforme a las convenciones aprobadas. Si se ejecuta alguna comprobación no interactiva de Spec Kit, debe estar documentada y fijada a una versión concreta. | Falta un artefacto obligatorio o una herramienta falla. No se deben inventar comandos de validación que Spec Kit no documente. |

| Etapa | Trabajo de Jenkins | Condición de bloqueo |
|---|---|---|
| Validar contrato de API | En proyectos de API, instalar dependencias desde el lockfile y ejecutar las validaciones acordadas con Spectral y Redocly: contrato modular, referencias, consolidación y contrato consolidado. | Hay errores bloqueantes, referencias irresolubles o falla la consolidación. |
| Publicar evidencias | Conservar informes, contrato consolidado cuando corresponda, revisión Git del consumidor, commits de fuentes, versiones de herramientas y huella del resultado. | No se puede asociar una evidencia al commit y las fuentes que la generaron. |

La etapa de verificación de Spec Kit no debe confundirse con ejecutar automáticamente el ciclo de autoría. Jenkins puede comprobar archivos versionados y ejecutar comprobaciones técnicas deterministas; no debe lanzar una interacción con un agente para aclarar o completar requisitos dentro de una build. Los nombres y argumentos concretos de comandos se incorporarán únicamente después de comprobarlos para las versiones fijadas y documentarlos en el repositorio.

### 1.7.2 Comportamiento por tipo de repositorio

- **`api-standards`:** valida la constitución corporativa y sus reglas de formato, identificadores y referencias. No necesita importar la constitución de dominio.
- **`domain-governance`:** recupera el commit corporativo declarado, compone y compara la constitución efectiva del dominio, y valida los recursos de gobierno del dominio. No genera ni modifica constituciones de APIs consumidoras.
- **Proyecto de API:** recupera los commits corporativo y de dominio fijados, compone y compara su constitución efectiva, verifica sus artefactos Spec Kit y valida su contrato OpenAPI y el consolidado.

Un cambio en `api-standards` ejecuta la pipeline de ese repositorio. Cuando el dominio quiera adoptar la nueva revisión, actualizará su referencia en un merge request propio y ejecutará su pipeline. Cada API actualizará después, si corresponde, su referencia al dominio y/o a la fuente corporativa mediante su propio merge request. Jenkins puede informar qué consumidores tienen una revisión anterior, pero no debe actualizar sus ramas, regenerar sus archivos ni aprobar esos cambios de forma automática.

### 1.7.3 Seguridad y reproducibilidad

- Usar credenciales de solo lectura para recuperar los repositorios fuente. Guardarlas en el almacén de credenciales de Jenkins y nunca imprimir secretos en consola ni incluirlos en artefactos.
- Ejecutar builds de merge requests con privilegios mínimos. No exponer credenciales de escritura a cambios no revisados ni a ramas de contribución que no sean de confianza.
- Fijar las versiones de Spec Kit, Node.js, Spectral, Redocly y de la imagen del agente cuando aplique. Instalar dependencias desde archivos de bloqueo y evitar descargas de versiones flotantes durante la build.
- Hacer que la composición sea determinista: mismas fuentes, commits y perfil deben producir exactamente la misma constitución efectiva. Si la organización no dispone de una composición reproducible, esa capacidad debe implementarse y probarse antes de exigirla en Jenkins.
- Conservar los informes incluso cuando una etapa posterior falle y marcar el resultado global como fallido ante cualquier control bloqueante. Una advertencia solo será no bloqueante si la política del proyecto lo establece explícitamente.
- Mantener la validación y consolidación separadas de la publicación en el gestor de APIs. Esta pipeline no debe desplegar servicios ni publicar contratos por el mero hecho de validar un merge request.

### 1.7.4 Puesta en marcha

1. Acordar el proyecto GitLab central que mantendrá el `Jenkinsfile`, su responsable, los agentes de ejecución, los permisos de lectura, la integración de webhooks y la política de disparo descrita arriba.
2. Implementar primero la recuperación de fuentes por commits exactos y la composición determinista de la constitución efectiva. Probarla desde un checkout limpio en un repositorio corporativo, uno de dominio y uno de API.
3. Añadir la comparación de la constitución generada con la versionada y hacer que una diferencia o referencia inválida bloquee la build.
4. Añadir las comprobaciones de artefactos Spec Kit y, para APIs, las validaciones de Spectral y Redocly usando las dependencias fijadas del proyecto.
5. Publicar las evidencias de una build correcta y de una build fallida; comprobar que ambas identifican el commit consumidor y los commits de gobierno utilizados.
6. Habilitar la política de protección para que los controles bloqueantes deban pasar antes de integrar cambios. Mantener la aprobación humana para cambios de gobierno, decisiones funcionales y publicación.

Esta guía define las responsabilidades y los controles, no declara implementada una pipeline. La selección de versiones, los comandos verificados, la composición de constituciones y la configuración real de Jenkins deben aprobarse y probarse en los repositorios del taller antes de afirmar que el flujo está operativo.

Referencias oficiales: GitHub Spec Kit y Jenkins Pipeline Syntax.

## 1.8 Comprobación rápida de dependencia

Antes de aprobar una constitución efectiva, el revisor debe poder responder afirmativamente a estas preguntas:

- ¿La fuente corporativa está identificada por una revisión aprobada y sus obligaciones permanecen completas?
- ¿La especialización del dominio cita los capítulos corporativos que concreta y añade solo reglas de su ámbito?
- ¿El perfil de API referencia la constitución del dominio propietario y limita sus decisiones a la API?
- ¿Cada repositorio tiene su propio proyecto Spec Kit y el documento efectivo está disponible localmente?
- ¿Las revisiones fijadas permiten reproducir la composición y detectar conflictos en Jenkins?
- ¿La constitución efectiva está generada desde las fuentes, y no editada manualmente para ocultar una discrepancia?

Si alguna respuesta es negativa, el gobierno aún no está listo para autorizar la especificación o la generación contractual con Spec Kit.